Relatório Fase 1 – Computação Paralela

Dinâmica Molecular

Pedro Lindo
PG65436
Universidade do Minho
Braga, Portugal
pg65436@alunos.uminho.pt

Gonçalo Silva
PG65412
Universidade do Minho
Braga, Portugal
pg65412@alunos.uminho.pt

***Abstract*— Este relatório descreve a otimização e paralelização em OpenMP de um simulador de dinâmica molecular para o gás Árgon. Através de profiling inicial, identificaram-se os hot-spots de complexidade O(N^2), que foram otimizados matematicamente. O paralelismo foi implementado contornando Data Races através do cálculo da matriz completa de interações. Os resultados no cluster Deucalion demonstram uma excelente escalabilidade forte, reduzindo o tempo de execução de 221.45s para 5.11s utilizando 48 threads.**

Keywords—Computação Paralela, Dinâmica Molecular, Open MP, escalabilidade forte, Deucalion .

I. Introdução e análise do caso de estudo

O programa em estudo realiza uma simulação de Dinâmica Molecular aplicada a um sistema de partículas de gás Árgon. Para representar as forças entre as partículas, utiliza o potencial de Lennard-Jones, enquanto a atualização das posições e velocidades ao longo do tempo é feita através do método de integração Velocity Verlet. O sistema considera ainda paredes elásticas como condições de fronteira.

Na configuração inicial, definida no ficheiro inputdata.txt, são consideradas 5000 partículas e 201 passos temporais, com uma temperatura de 100 K e uma densidade de 35000 mol/m³.

A análise das dependências do ciclo temporal principal (main) permite concluir que cada iteração depende dos valores das posições, velocidades e acelerações obtidos na iteração anterior. Por esse motivo, o ciclo temporal externo tem de ser executado de forma sequencial. Assim, a possibilidade de explorar o paralelismo de dados (data parallelism) encontra-se sobretudo nos ciclos internos, que são executados em cada passo da simulação.

II. Perfilamento inicial e hot-spots

A análise inicial do desempenho da versão sequencial foi realizada num nó de computação ARM do cluster Deucalion, equipado com um processador Fujitsu A64FX de 48 cores e 32 GB de memória HBM2. Foram utilizados o compilador GCC/15.2.0 e a ferramenta perf record, considerando 200 partículas

O perfil de execução identificou dois pontos críticos (hot-spots), ambos com complexidade quadrática O(N²):

- Função Potential: Representa 65,4% do tempo de execução, sendo responsável pelo cálculo da energia potencial entre todos os pares de partículas diferentes (i, j).
- Função computeAccelerations: Corresponde a 33,1% do tempo de CPU e calcula as forças entre pares de partículas, considerando apenas os casos em que j é superior a i.

No conjunto, estas duas funções representam mais de 98,5% do tempo total de CPU. As restantes rotinas, VelocityVerlet, MeanSquaredVelocity e Kinetic, apresentam complexidade linear O(N) e representam menos de 1,5% do tempo de execução.

III. Otimização e estratégia de paralelização OpenMP

Para melhorar o desempenho nos nós ARM, foram aplicadas duas abordagens aos pontos críticos identificados:

A. Otimização matemática e de compilação

Nos ciclos internos das funções Potential e computeAccelerations, as chamadas repetidas a pow() e sqrt() foram substituídas por operações algébricas mais simples, utilizando o inverso da distância ao quadrado (inv_r2 = 1.0 / rSqd). O programa foi também compilado com as flags -O3 -ffast-math -fopenmp -march=native -g -fno-omit-frame-pointer. Após estas alterações, o perfil obtido com perf confirmou a eliminação do principal gargalo de Potential, passando computeAccelerations a representar 48,33% do tempo de execução.

B. Paralelização e resolução de Data Races

A função Potential pode ser paralelizada diretamente através de reduction(+:Pot), uma vez que cada iteração contribui apenas para a soma da energia potencial [cite: 1, 29]. Já em computeAccelerations, o ciclo triangular (j > i) atualiza simultaneamente a[i][k] e a[j][k], podendo originar condições de corrida (Data Races) entre threads.

Foram analisadas três alternativas para resolver este problema:

1. Sincronização explícita (#pragma omp atomic): Evita conflitos nas escritas em a[j][k], mas introduz custos de sincronização entre threads.
2. Acumulação privada: Mantém o ciclo triangular sem conflitos, mas exige memória adicional por thread e uma fase de combinação dos resultados.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

3. Matriz completa de interações (N x N): Cada thread calcula independentemente as forças sobre a partícula i, escrevendo apenas em a[i][k], sem recorrer à simetria da 3.ª Lei de Newton.

Foi escolhida a terceira abordagem, utilizando #pragma omp parallel for schedule(dynamic, 16) nas duas funções. Apesar de duplicar o número de operações aritméticas, esta solução elimina as condições de corrida e a necessidade de sincronização intermédia, permitindo distribuir o trabalho pelos 48 cores.

IV. Análise de escalabilidade forte e resultados

Para avaliar a escalabilidade forte da solução, manteve-se fixa a dimensão do problema (5000 partículas e 201 passos temporais) e variou-se o número de threads entre 1, 2, 4, 8, 16, 32 e 48, num nó ARM do Deucalion.

Em todas as execuções, os resultados termodinâmicos mantiveram-se iguais: temperatura média de 131,45506 K, pressão média de 131001228,45076 Pa e fator de compressibilidade Z = 3,42449, confirmando a consistência dos resultados.

A Tabela I apresenta os tempos de execução (wall-clock time), o Speedup ($S_p = T_1 / T_p$) e a eficiência ($E_p = S_p / p$) obtidos.

TABLE I. Tempo de execução, Speedup e eficiência (n=5000)

Threads	Tempo (s)	Speedup	Eficiência
1	221.45	1.00x	100.0%
2	112.90	1.96x	98.1%
4	58.61	3.78x	94.5%
8	28.56	7.75x	96.9%
16	14.29	15.50x	96.9%
32	7.18	30.84x	96.4%
48	5.11	43.34x	90.3%

Os resultados mostram um crescimento quase linear do Speedup até às 32 threads, com uma eficiência superior a 96%. Utilizando os 48 cores do processador A64FX (OMP_NUM_THREADS=48), o tempo de execução passou de 221,45 s para 5,11 s, atingindo um Speedup de 43,34x e uma eficiência de 90,3%.

A ligeira redução da eficiência entre 32 e 48 threads deve-se às partes do programa que continuam a ser executadas sequencialmente, nomeadamente a atualização das posições e velocidades e a escrita dos ficheiros de saída, bem como ao custo de gestão das barreiras OpenMP. Com a redução do tempo total de execução, estes fatores passam a ter maior impacto no desempenho.

Conclusão

A metodologia utilizada permitiu identificar que mais de 98,5% do tempo de computação estava concentrado nas funções de cálculo do potencial e das forças, ambas com complexidade $O(N^2)$. A simplificação algébrica e a paralelização com OpenMP permitiram aproveitar os 48 cores do cluster Deucalion, reduzindo o tempo de execução em 97,7% e alcançando um Speedup de 43,34x, sem comprometer os resultados da simulação